AUTONOMOUS ENGINEERING ARCHITECTURE

SubAgent 추가에 따른 시스템 연쇄 영향도 분석 (Rule, AGENTS.md, Skill 변경 및 통합)

독립 서브에이전트 도입 시 프로젝트 거버넌스, 지침, 룰, 실행 인터페이스의 상호작용 체계 분석

프로젝트	문서 성격	발행 형식	일자
FC (Unreal Engine 5.8)	아키텍처 연쇄 수정 영향도 분석 보고서	Executive PDF Edition	2026. 10. 28

Executive Summary
Antigravity 에이전트 환경에서 **새로운 SubAgent(예: `adversarial-auditor`)**를 추가하는 작업은 단순한 프롬프트 정의에 그치지 않습니다. 에이전트 간의 권한 분리, 엄격한 심사 루브릭, 자율 파이프라인 연계, 슬래시 명령어 트리거를 유기적으로 완성하기 위해 ① **Rule 계층**, ② **거버넌스 계층(AGENTS.md)**, ③ **Skill 계층**이 반드시 동기화되어 수정되어야 합니다. 본 보고서는 서브에이전트 도입 시 이 4대 계층이 어떻게 상호작용하고 변경되는지를 체계적으로 규명합니다.

1. SubAgent 추가 시 4계층 연쇄 변경 개요 (4-Tier Co-Evolution)

AI 하네스에서 서브에이전트가 단독으로 생성되면, 메인 에이전트는 해당 서브에이전트의 존재를 인지하거나 언제 호출해야 할지 알 수 없으며, 서브에이전트 또한 프로젝트 고유의 설계 원칙을 준수할 수 없습니다. 따라서 다음 4단계 연쇄 수정이 수반됩니다:



2. 각 계층별 세부 변경 내역 및 설계 의도

각 계층이 왜 수정되어야 하며, 어떤 엔지니어링 의도를 담고 있는지 상세 분석합니다.

2.1 SubAgent Layer (정의 및 격리)

- **변경 내용**: `define_subagent` API를 통해 `adversarial-auditor` 등록.
- **핵심 설계 의도 (Read-Only)**: `EnableWriteTools: false`를 강제하여, 심사 서브에이전트가 코드를 직접 수정하는 과정에서 발생할 수 있는 환각(Hallucination) 및 2차 버그 유발을 원천 차단했습니다.
- **모델 다변화**: `Model: pro`를 지원하여, 코드를 작성한 모델보다 더 깊은 추론력을 가진 상위 모델이 비판적 결함을 찾아내도록 구성.

2.2 Rule Layer: `.antigravity/rules/` 의 확장

- **신규 추가 파일**: `.antigravity/rules/07-adversarial-audit-standards.md`
- **왜 서브에이전트 프롬프트에 전부 넣지 않고 Rule 파일로 분리하는가?**
  - **토큰 효율성 (Progressive Disclosure)**: 심사 기준 전체를 시스템 프롬프트에 상시 유지하면 서브에이전트 구동 시마다 막대한 컨텍스트 토큰이 소모됩니다. 기준을 외부 룰 파일로 모듈화하면 필요할 때만 참조할 수 있습니다.
  - **일관성 및 단일 진실 공급원(SSOT)**: 인간 개발자, 메인 에이전트, 심사 서브에이전트가 모두 **동일한 5대 심사 기준 문서**를 읽고 소통하므로 피드백의 왜곡이 발생하지 않습니다.
- **5대 심사 차원의 확립**:
  1. **코드 간결성**: 불필요한 보일러플레이트 및 과도한 클래스 래핑 배제.
  2. **기존 구조 및 코드 활용**: 기존 Subsystem, DataTable, Component 재사용 강제.
  3. **범위 내 적정 확장성**: 기획서에 명시된 범위를 초과하는 미래 대비 과설계(YAGNI 위반) 억제.
  4. **네트워크 최적화**: HasAuthority, RPC Connection 소유권, 복제 조건(COND) 엄수.
  5. **일반 성능 최적화**: Tick 기본 비활성화, 핫패스 힙 할당 제로화, TObjectPtr GC 위생.

2.3 Governance Layer: `AGENTS.md` 의 동기화

- **수정 위치**: `AGENTS.md` Section 3(파이프라인 실행 모드) 및 Section 4(도메인 룰 목록).
- **변경 내역**:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          PIPELINE EXECUTION MODES                        │
├──────────────────────────────────────────────────────────────────────────┤
│ ...                                                                      │
│ 7. Adversarial Audit (/interrogate): Read-only subagent review on 5 core dimensions │
│ 8. Full Release (`full`)      : Sequential (Static -> Build -> Test -> [Auto-Commit])│
└──────────────────────────────────────────────────────────────────────────┘
```

- **설계 의도**:
  - `AGENTS.md`는 세션이 새로 열리거나 다른 개발자가 참여할 때 가장 먼저 주입되는 프로젝트 헌법입니다.
  - 여기에 서브에이전트 검사 모드가 누락되면, 메인 에이전트는 코드를 작성한 후 서브에이전트에게 검사를 요청해야 한다는 사실 자체를 잊어버리게 됩니다. 따라서 프로토콜 가이드라인에 **의무적 교차 검사 절차**로 공식 편입했습니다.

2.4 Skill Layer: `.agents/skills/interrogate/` 의 구축

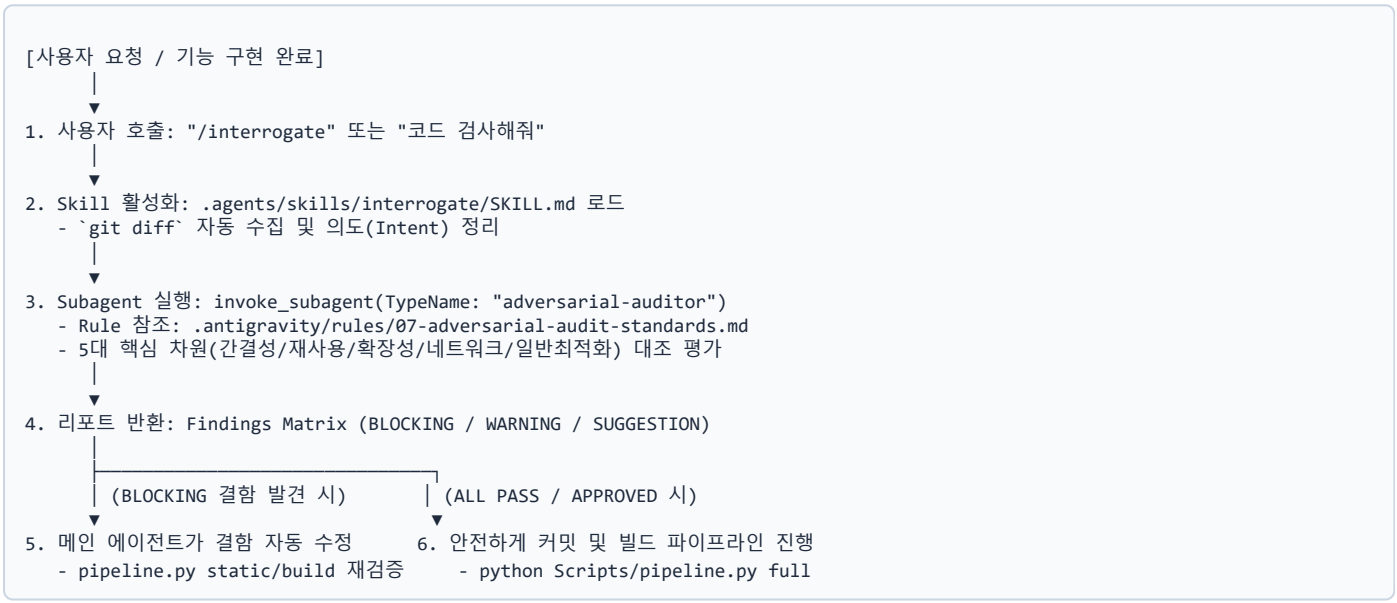
- **신규 추가 파일**: `.agents/skills/interrogate/SKILL.md`
- **역할 및 트리거링**: 사용자가 `/interrogate` 또는 "코드 검사해줘", "서브에이전트 리뷰 돌려줘"라고 말할 때 Antigravity 하네스가 해당 스킬을 즉시 감지하여 절차를 실행합니다.
- **자동화된 3단계 워크플로우**:
  1. **컨텍스트 취합**: `git diff HEAD`를 자동으로 수집하고 작성자의 구현 의도(Stated Intent)를 요약.
  2. **서브에이전트 격리 실행**: `invoke_subagent`를 통해 `adversarial-auditor`를 호출하여 심사 의뢰.
  3. **판정 및 보완**: 반환된 Findings Matrix(BLOCKING / WARNING / SUGGESTION)를 확인하여 치명적 결함을 우선 수정.

> **핵심 결론**: 서브에이전트는 독립된 스크립트가 아니라 **[룰 - 거버넌스 - 스킬 - 런타임]**의 4중 결합체로 구성될 때 비로소 사람의 개입 없이도 누락 없이 안정적으로 작동합니다.

3. 서브에이전트 추가에 따른 종합 시스템 영향도 매트릭스

계층 / 파일	유형	주요 변경 및 추가 내용	상호작용 및 파급 효과
`define_subagent (adversarial-auditor)`	SubAgent	Read-Only 심사 전문 에이전트 런타임 정의, 엄격한 심사관 시스템 프롬프트 부여	코드 오염 없이 비판적 결함만 탐지하는 독립 실행 주체 생성
`.antigravity/rules/07-adversarial-audit.md`	Rule	간결성, 기존 구조 활용, 범위 내 확장성, 네트워크/일반 최적화 5대 루브릭 규정	심사관과 작업자가 동일한 잣대로 소통할 수 있는 객관적 기준 제공
`AGENTS.md (Section 3 & 4)`	Governance	파이프라인 실행 모드 7번에 편입, 도메인 룰 목록 링크, 최소 수정 원칙 준수 강조	메인 에이전트가 코드 커밋 전 서브에이전트 감사를 의무화하도록 강제
`.agents/skills/interrogate/SKILL.md`	Skill	/interrogate 슬래시 커맨드 등록, diff 수집 -> 서브에이전트 호출 자동화	사용자 한마디로 복잡한 멀티에이전트 교차 검증 절차가 원클릭 실행됨

4. 엔드투엔드 실행 라이프사이클 (End-to-End Operational Lifecycle)



5. 향후 신규 SubAgent 확장 시 표준 체크리스트

향후 프로젝트에 또 다른 서브에이전트(예: *Balance-Sim-Auditor, Asset-Validator* 등)를 추가할 때도 이번과 동일하게 다음 4단계를 수행해야 시스템 무결성이 유지됩니다:

1. **SubAgent 정의**: `define_subagent`로 목적에 맞는 도구 권한(Read-Only 여부)과 시스템 프롬프트 설정.
2. **도메인 룰 분리**: `.antigravity/rules/XX-*.md`에 점진적 공개(Progressive Disclosure)가 가능한 기준 문서 작성.
3. **지침서 반영**: `AGENTS.md`의 파이프라인 모드 및 룰 목록에 공식 등록하여 단일 진실 공급원 유지.
4. **호출 스킬 구축**: `.agents/skills/<name>/SKILL.md`에 트리거 키워드와 컨텍스트 조립 절차를 작성.